

Redes de Computadoras

Laboratorio - Parte 1 - Programación con Sockets

Se debe implementar en Python una aplicación que, mediante *sockets*, permita enviar y recibir *mensajes* y *archivos* entre pares. Es decir, la aplicación debe poder recibir mensajes y archivos en todo momento, así como también enviarlos a otra instancia de la aplicación vía *sockets*. Para ello, el proceso deberá bifurcarse o utilizar hilos, de modo que una parte se encargue de la recepción y otra de la transmisión. Los mensajes tendrán una longitud máxima predefinida.

```
MAX_LARGO_MENSAJE = 255
```

El programa será invocado de la siguiente manera:

```
python mensajeria.py port ipAuth portAuth
```

Levantará el argumento *port*, que será el puerto donde oír el *receptor*, y escribirá el *emisor* y los argumentos *ipAuth* y *portAuth* que indicarán la dirección del servidor de autenticación.

El programa constará de dos grandes partes, una *emisora* y otra *receptora*.

• Receptora:

Esta oír continuamente por un *puerto* que se le pasará por argumento al programa, al llegar los *mensajes* a ese *puerto*, los imprimirá en pantalla (*salida estándar*), junto con la *fecha y hora*, y el *nombre de usuario* del emisor, los mensajes serán *strings* que tendrán el *ip* del emisor, seguido de un espacio seguido del *nombre de usuario*, seguido de un espacio, seguido de “*dice:* ” y el *mensaje*, estos mensajes, como ya se dijo, tendrán un largo máximo.

Ejemplos de impresión del receptor:

```
[2026.06.23 17:02] 192.168.33.15 nwirth dice: Feliz Cumple!!!!
```

Al llegar archivos, los guardará en el directorio “*actual*” del proceso en el *FileSystem* local.

Ejemplos de impresión del receptor:

```
[2026.06.23 17:05] 192.168.33.11 <Recibido ./foto.jpg de nwirth>
[2026.06.23 17:05] 192.168.33.11 <Error Recibiendo Archivo de nwirth>
```

• Emisora:

Esta enviará mensajes a un determinado *receptor* al puerto pasado como parámetro, levantará de la entrada estándar el destino del mensaje (*ip o nombre de host*) y el contenido, por ejemplo:

```
192.168.33.15 Feliz Cumple!!!!
tecnoinf315.esi.edu.uy Feliz Cumple!!!!
tecnoinf315 Feliz Cumple!!!!
```

Si en lugar de poner un *ip o nombre de host* se pone un “*” se debe enviar el mensaje a todos los receptores presentes en la red del emisor.

```
* Gracias a todos los que se acordaron
```

Cuando un mensaje es enviado a todos todos (*Broadcast*) el *receptor* también debe imprimirlo (como con el resto de los mensajes).

Si en lugar de un mensaje se pone un "&file" y un "path" se debe enviar el archivo que está en *path* en el FS local.

Ejemplo de impresión del emisor:

```
* &file ./foto.jpg
192.168.33.15 &file ./foto.jpg
tecnoinf315.esi.edu.uy &file ./foto.jpg
tecnoinf315 &file ./foto.jpg
```

Ambas partes del programa escribirán en la salida estándar, por lo que en pantalla aparecerán mezcladas las salidas del *emisor* y del *receptor*.

• Autenticación

Los usuarios del programa deben autenticarse, para ello se dispondrá de un ejecutable *redes-auth* este será invocado de la siguiente manera:

```
./redes-auth port
```

Este programa autenticará sobre un archivo de texto con el siguiente formato, cada línea será de la forma:

```
usuario-contraseña_codificada_en_md5;Nombres_Apellidos;
```

por ejemplo (*la contraseña es el nombre de usuario en md5*):

```
aturing-50bc36a72e099d0ae1e78186a0859d46;Alan_Mathison_Turing;
alovelace-fd3b63b26dfd701c68eef561f464691e;Augusta_Ada_Byron;
jkonvalina-5fc2f043528bd2998123667d58f7a065;John_Konvalina;
jvneumann-941a833c2db9668cd995c74a3c8f3985;Neumann_Janos_Lajos;
nwirth-95f845845deacdf77cf85be58b5d744a;Niklaus_Wirth;
cbabbage-b6fe35296e44d8d2955ef7f609f90904;Charles_Babbage;
edijkstra-cc38c2a1e714e20bc9ffa725614e810a;Edsger_Wybe_Dijkstra;
```

El servidor al conectarse un cliente le responderá:

```
Redes 2026 - Laboratorio - Autenticación de Usuarios
```

Los clientes enviarán lo siguiente:

```
usuario-contraseña_codificada_en_md5
```

El servidor de autenticación responderá en caso afirmativo

```
SI
Nombres_Apellidos
```

Y en caso negativo:

```
NO
```

Los fines de línea enviados (y recibidos) por el servidor se codifican CRLF (\r\n)

- **¿IP o Nombre de Host?**

Los usuarios pueden optar por enviar los mensajes usando el *ip* del destino, o en su defecto el *nombre del host* destino. Se debe tener en cuenta que los paquetes van con el *ip*, por lo que en el caso que se escriba el *nombre de host*, este se debe resolver previo a mandar el mensaje.

Ejemplo de sesión del servidor de autenticación:

```
> Redes 2026 - Laboratorio - Autenticación de Usuarios
< alovelace-bd5f3cbebf74ce4f5126e4536fdacb80
> SI
> Augusta_Ada_Byron
```

(los “<” y “>” no forman parte de la salida, indican si son mensajes entrantes o salientes)

Ejemplo de sesión del programa de mensajería:

```
./mensajeria 22764 172.23.1.240 3456
Usuario: aturing
Clave: *****
Bienvenido Alan_Mathison_Turing
[2026.06.23 17:02] 192.168.33.15 nwirth dice: Feliz Cumple!!!!
192.168.33.15 Gracias!!!!
[2026.06.23 17:03] 192.168.33.15 nwirth dice: Merece!!!!
192.168.33.11 No me tenés que decir nada hoy??
[2026.06.23 17:05] 192.168.33.11 edijkstra dice: Por?
192.23.33.11 Es mi cumple!!!
[2026.06.23 17:06] 192.168.33.11 edijkstra dice: ahh que boludo!!
[2026.06.23 17:06] 192.168.33.11 edijkstra dice: me pensaba que era en mayo!!!
[2026.06.23 17:07] 192.168.33.11 edijkstra dice: Feliz Cumple!!!
* Gracias a todos los que se acordaron
[2026.06.23 17:08] 192.168.33.10 aturing dice: Gracias a todos los que se acordaron
CTRL + C Recibido.... Cerrando Sesión
```

```
./mensajeria 22764 192.168.1.240 3456
Usuario: aturing
Clave: *****
Bienvenido Alan_Mathison_Turing
192.168.33.11 &file ./foto.jpg
192.168.33.11 Te llegó el archivo???
[2026.06.23 18:06] 192.168.33.11 nwirth dice: Si, impecable!!!!
CTRL + C Recibido.... Cerrando Sesión
```

El programa deberá atender las señales y alarmas del sistema, y se terminará al recibir un CTRL + C o recibir un KILL o TERM. Al recibir estas señales deberá liberar de manera correcta los recursos que haya solicitado, *sockets* memoria, etc.

El comportamiento de entradas y salidas debe ser el que figura en los ejemplos.

Sobre la Entrega:

- El trabajo es obligatorio.
- Se pueden obtener hasta 20 puntos (entre este trabajo y la parte 2).
- El trabajo se realizará en grupos de hasta 4 personas.
- Se deben inscribir los grupos enviando un mail a mzabalza@fing.edu.uy indicando los integrantes datos (CI, Nombre y Apellido). Plazo hasta el *lunes 18 de mayo inclusive*. Quienes no se inscriban se considerará que desisten de hacer el obligatorio con la consecuente pérdida del curso.
- Se deberá entregar:

- `mensajeria.py`

Este contendrá la solución al problema y un comentario con los nombres y números de documento de todos los integrantes del grupo.

- `comentarios.txt`

Contendrá comentarios sobre la solución, errores, etc.

- Estos archivos deberán venir comprimidos en uno que se llamará:
 - `redes-lab.tar.gz`
- Hay tiempo hasta el día **domingo 7 de junio** a las 23:59 UYT.
- Se dispondrá de un entregador en el Campus Virtual.
- Se coordinará una pequeña defensa con **todos** los integrantes del grupo.

- ***Lecturas Recomendadas:***

- ***Bifurcación:***

- <https://docs.python.org/es/3.8/library/threading.html>

- ***Programación Básica con Sockets:***

- <https://docs.python.org/es/3/howto/sockets.html>
 - <https://docs.python.org/3/library/socket.html>
 - <https://realpython.com/python-sockets/>

- ***Señales:***

- <https://old.chuidiang.org/clinux/senhales/senhales.php>
 - <https://docs.python.org/es/3/library/signal.html>